САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ, МЕХАНИКИ И ОПТИКИ

ФАКУЛЬТЕТ ПРИКЛАДНОЙ ИНФОРМАТИКИ

Отчет по лабораторной работе №2

по дисциплине «Методы оптимизации»

Тема: «Метод ломаных»

Выполнил:
Бен Шамех Абделазиз
Группа: К3339
Поток: МетОпт 1.2
Преподаватель: Савелий Лаврентьев

Санкт-Петербург
2026 г.

Постановка задачи

Нужно написать программу, которая ищет глобальный минимум одномерной липшицевой функции на заданном отрезке методом ломаных (методом Пиявского). На вход подаются функция в виде строки, концы отрезка [a, b] и точность ε. Программа должна построить график функции, вспомогательных функций и итоговой ломаной, выдать приближённые значения аргумента и минимума, число итераций и затраченное время.

Работу нужно продемонстрировать на тестовых функциях хотя бы с двумя локальными минимумами на отрезке (Растригин и другие). Дополнительно к основной части выполнены два пункта: определение константы Липшица через вторую производную с объяснением принципа и подкаст-рефлексия (аудиофайл приложен к работе в Moodle отдельно).

1. Теоретические сведения

Функция W(x) называется липшицевой на [a, b], если найдётся такое L > 0, что для любых u, v из отрезка |W(u) − W(v)| ≤ L·|u − v|. Наименьшее такое L и есть константа Липшица; на практике подойдёт любая оценка сверху.

Идея метода в том, что минимум ищется не у самой функции, а у ломаной, которая всё время лежит под графиком. Для каждой уже вычисленной точки u_i строится «галочка»

$$g_i(x, u_i) = W(u_i) − L·|x − u_i|,$$

а ломаная порядка n — это максимум всех галочек:

$$p_n(x) = max_{i=0..n}\, g_i(x, u_i).$$

Из условия Липшица следует $p_n(x) \le W(x)$ на всём отрезке, то есть ломаная — миноранта. Верхние вершины ломаной лежат на графике W, нижние — между ними, под графиком.

Алгоритм. На стартовых итерациях берём $u_0 = a$ и $u_1 = b$ и считаем W в этих точках. Дальше на каждой итерации n находим $u_n = \operatorname{argmin} p_{n-1}(x)$, вычисляем $W(u_n)$, строим новую галочку и обновляем ломаную. Минимум ломаной достигается в одной из нижних вершин. Нижняя вершина между соседними верхними $u_q < u_v$ — это пересечение их галочек:

$$u_{cross} = (W(u_q) − W(u_v)) / (2L) + (u_q + u_v) / 2,$$
$$p(u_{cross}) = (W(u_q) + W(u_v)) / 2 − L·(u_v − u_q) / 2.$$

Поэтому на каждой итерации достаточно пройтись по соседним парам верхних вершин, выбрать пару с наименьшим p и взять её u_{cross} в качестве следующей

точки.

Остановка: процесс заканчивается, когда $W(u_n) - p_{n-1}(u_n) < \varepsilon$ или исчерпан лимит итераций. Здесь $p_{n-1}(u_n)$ служит нижней оценкой глобального минимума, а лучшее найденное значение не больше $W(u_n)$. Поэтому **при условии, что L не меньше настоящей константы Липшица**, найденное значение отличается от истинного минимума меньше чем на ε. Если L занижена, такой гарантии нет: для L, найденной через вторую производную, условие выполнено (с точностью до ошибок округления), а для оценки по сетке и для L, введённой вручную, гарантии нет.

Неравенство «разность меньше ε» само по себе проверять недостаточно, потому что оно выполняется и для отрицательной разности. Если $W(u_n)$ оказалось *ниже* $p_{n-1}(u_n)$, то ломаная лежит выше графика функции, а миноранта этого допускать не может. Единственное объяснение – L меньше настоящей константы Липшица. В этом случае программа не сообщает об успехе, а останавливается со статусом «L занижена» и показывает, какой наклон функции она обнаружила (раздел 5).

2. Константа Липшица через вторую производную

Для гладкой функции подходит $L = \max |W'(x)|$ на [a, b]. Чтобы найти этот максимум без перебора значений по мелкой сетке, используется вторая производная. Функция W' непрерывна на отрезке, поэтому её модуль достигает максимума либо на концах, либо во внутренней точке, где W' имеет экстремум, то есть где $W''(x) = 0$. Значит, достаточно проверить конечное число кандидатов: концы a, b и корни уравнения $W''(x) = 0$.

Порядок действий в программе: (1) sympy берёт символьные производные W' и W'' от введённой строки; (2) на отрезке ищутся промежутки, где W'' меняет знак (сетка нужна только для этого поиска), и корни в них уточняются методом Брента до 10^{-14}; (3) в каждом кандидате считается $|W'|$, берётся наибольшее значение, оно и есть L. Саму W' мы вычисляем точно, поэтому коэффициент запаса не нужен.

Пример: $W(x) = x + \sin(\pi x)$ на $[-2; 2]$. $W' = 1 + \pi\cdot\cos(\pi x)$, $W'' = -\pi^2\cdot\sin(\pi x)$. Нули W'' — точки $x = -1, 0, 1$, плюс концы ± 2. Значения $|W'|$: в $x = \pm 2$ и $x = 0$ получается $1 + \pi \approx 4{,}1416$, в $x = \pm 1$ получается $\pi - 1 \approx 2{,}1416$. Итак $L = 1 + \pi \approx 4{,}1416$.

Способ через вторую производную работает только если W'' существует и вычисляется на всём отрезке. Для функций с изломом, например $|x - 1|$, вторая

производная содержит разрыв, и sympy не может её вычислить. Тогда программа выводит заметное предупреждение (отдельный блок из восклицательных знаков), переходит на оценку по сетке и в итоговом результате отдельной строкой пишет, что гарантии точности в этом режиме нет (раздел 5).

Для сравнения в программе оставлен и численный способ: максимум $|\Delta W/\Delta x|$ по сетке из 5000 точек, умноженный на запас 1,2. Он не требует символьных производных, но даёт завышенное L, а чем больше L, тем более пологи нижние вершины и тем больше итераций нужно.

Таблица 1 – Сравнение двух способов оценки константы Липшица

Функция	**ε**	**L (2-я произв.)**	**L (сетка×1,2)**	**Итераций (2-я)**	**Итераций (сетка)**
Растригин	0,001	67,3327	80,7988	357	472
Из задания	0,01	4,1416	4,9699	11	14
Гриванк	0,001	1,0071	1,2085	208	280

Видно, что точная константа заметно сокращает число итераций (в среднем на четверть) при том же результате.

3. Программная реализация

Программа написана на Python (numpy, matplotlib, sympy, scipy) и состоит из нескольких функций. Тестовые функции хранятся в списке TESTS, новую можно добавить одной строкой.

g и p – галочка и ломаная

g строит галочку с наклоном ±L из точки u; значение W(u) передаётся готовым, чтобы функция не пересчитывалась. p – ломаная как максимум всех галочек, в основном цикле она не нужна, используется для графика.

```
def g(x, u, W_u, L):
    """ «галочка» g(x, u) = W(u) - L*|x - u| """
    return W_u - L * abs(x - u)

def p(x, u_list, W_list, L):
    """ ломаная p_n(x) = max_i g(x, u_i) """
    return max(g(x, u, w, L) for u, w in zip(u_list, W_list))
```

Рисунок 1 – Функции g и p

find_min_p и LipschitzViolation – поиск нижней вершины и контроль L

Для каждой пары соседних верхних вершин по формулам из теории считаются точка пересечения галочек и значение ломаной в ней. Выбирается пара с наименьшим значением, поэтому итерация стоит O(n). Перед расчётом проверяется, что наклон |ΔW/Δu| между соседями не больше L. Если это не так, точка пересечения вышла бы за границы интервала, и функция выбрасывает исключение LipschitzViolation. Раньше такая точка молча возвращалась внутрь интервала, и ошибка в L скрывалась.

```
class LipschitzViolation(Exception):
    """ найдены соседние точки с |W(u_j) - W(u_i)| > L*(u_j - u_i): L занижена """

    def __init__(self, u1, u2, slope):
        super().__init__(f"наклон на [{u1:.6g}; {u2:.6g}] равен {slope:.6g}")
        self.u1, self.u2, self.slope = u1, u2, slope
```

Рисунок 2 – Исключение LipschitzViolation

```
def find_min_p(u_list, W_list, L):
    """
    Ищем u_n = argmin p_{n-1}(x).

    Минимум ломаной лежит в одной из нижних вершин, а нижняя вершина - это
    точка пересечения галочек двух соседних верхних вершин u_i и u_{i+1}:
        u_cross = (W_i - W_{i+1}) / (2L) + (u_i + u_{i+1}) / 2
        p(u_cross) = (W_i + W_{i+1}) / 2 - L*(u_{i+1} - u_i) / 2
    Значение в нижней вершине считаем сразу по формуле, поэтому за итерацию
    достаточно одного прохода по интервалам.

    Если L настоящая, u_cross всегда попадает в [u_i, u_{i+1}]. Выход за
    границы интервала означает |W_i - W_{i+1}| > L*(u_{i+1} - u_i), то есть L
    занижена. Раньше такую точку молча «подтягивали» к интервалу, теперь
    выбрасывается LipschitzViolation.
    """
    best_u, best_val = None, float("inf")

    for i in range(len(u_list) - 1):
        ui, uj = u_list[i], u_list[i + 1]
        Wi, Wj = W_list[i], W_list[i + 1]

        slope = abs(Wi - Wj) / (uj - ui)
        if slope > L * (1 + TOL) + 1e-12:
            raise LipschitzViolation(ui, uj, slope)

        u_cross = (Wi - Wj) / (2 * L) + (ui + uj) / 2
        val = (Wi + Wj) / 2 - L * (uj - ui) / 2

        if val < best_val:
            best_u, best_val = u_cross, val

    return best_u, best_val
```

Рисунок 3 – Функция find_min_p

pyavsky – основной цикл

Стартуем с $u_0 = a$ и $u_1 = b$. На каждой итерации находим нижнюю вершину, вычисляем в ней W, вставляем новую точку в отсортированный список и проверяем условия. Причина остановки записывается в поле status: converged (точность достигнута), constant (функция постоянна), max_iter (лимит итераций), L_too_small (L занижена), bad_value (функция вернула NaN или бесконечность). Успехом считаются только первые два статуса. Отдельно разобран случай L = 0, при котором формулы с делением на 2L неприменимы.

```
def pyavsky(W, a, b, eps, L, max_iter=MAX_ITER):
    """
    Метод ломаных на [a, b]. Останов по W(u_n) - p_{n-1}(u_n) < eps.

    Причина остановки записывается в result['status']:
      converged  - точность eps достигнута (success = True)
      constant   - L = 0, функция постоянна, любая точка подходит (success = True)
      max_iter   - исчерпан лимит итераций, точность НЕ достигнута
      L_too_small- обнаружено, что L меньше настоящей константы Липшица
      bad_value  - функция вернула NaN, бесконечность или ошибку
    """
    t_start = time.perf_counter()

    u_list = [a, b]
    try:
        W_list = [float(W(a)), float(W(b))]
    except Exception as err:
        raise ValueError(f"не удалось вычислить W на концах отрезка: {err}")
    if not all(math.isfinite(w) for w in W_list):
        raise ValueError("значение функции на конце отрезка не конечно (NaN или inf)")

    def finish(status, message, iterations, gap, p_val):
        best = int(np.argmin(W_list))
        return {
            "u_list": u_list, "W_list": W_list,
            "u_min": u_list[best], "W_min": W_list[best],
            "lower_bound": p_val, "gap": gap,
            "iterations": iterations,
            "time": time.perf_counter() - t_start,
            "status": status,
            "success": status in ("converged", "constant"),
            "message": message,
        }

    # L = 0: формулы с делением на 2L не работают, разбираем случай отдельно
    if L <= L_MIN:
        mid = (a + b) / 2
        try:
            vals = W_list + [float(W(mid))]
        except Exception:
            vals = [0.0, 1.0]
        scale = 1 + max(abs(v) for v in vals)
        if max(vals) - min(vals) <= TOL * scale:
            value = W_list[0]
            return finish("constant",
                          "L = 0: функция постоянна на отрезке, минимум достигается в любой точке "
                          "(взят левый конец)", 0, 0.0, value)
        return finish("L_too_small",
                      "L = 0, но значения функции в трёх точках разные: функция не постоянна, "
                      "L нужно увеличить", 0, float("inf"), -float("inf"))
```

Рисунок 4 – Функция pyavsky, начало: проверки и случай L = 0

```
    iterations, gap, p_val = 0, float("inf"), -float("inf")

    while iterations < max_iter:
        iterations += 1

        try:
            u_new, p_val = find_min_p(u_list, W_list, L)
        except LipschitzViolation as v:
            return finish("L_too_small",
                          f"L = {L:.6g} занижена: на отрезке [{v.u1:.6g}; {v.u2:.6g}] наклон функции "
                          f"{v.slope:.6g} больше L. Настоящая константа Липшица не меньше "
                          f"{v.slope:.6g}", iterations, gap, p_val)

        try:
            W_new = float(W(u_new))
        except Exception as err:
            return finish("bad_value", f"не удалось вычислить W({u_new:.6g}): {err}",
                          iterations, gap, p_val)
        if not math.isfinite(W_new):
            return finish("bad_value", f"W({u_new:.6g}) = {W_new}: значение не конечно",
                          iterations, gap, p_val)

        # новая верхняя вершина - вставляем с сохранением порядка по x
        idx = bisect.bisect_left(u_list, u_new)
        u_list.insert(idx, u_new)
        W_list.insert(idx, W_new)

        gap = W_new - p_val

        # W(u_n) ниже ломаной: миноранта оказалась выше функции, значит L занижена.
        # Просто «gap < eps» здесь нельзя - отрицательный gap тоже меньше eps.
        if gap < -TOL * (1 + abs(W_new)):
            return finish("L_too_small",
                          f"L = {L:.6g} занижена: W(u_n) = {W_new:.6g} оказалось ниже ломаной "
                          f"({p_val:.6g}), а миноранта не может лежать выше функции",
                          iterations, gap, p_val)

        if gap < eps:
            return finish("converged",
                          f"достигнута точность: W(u_n) - p = {gap:.3e} < eps = {eps:g}",
                          iterations, gap, p_val)

    return finish("max_iter",
                  f"исчерпан лимит {max_iter} итераций, достигнута только W(u_n) - p = {gap:.3e} "
                  f"(нужно < {eps:g})", iterations, gap, p_val)
```

Рисунок 5 – Функция pyavsky, основной цикл и причины остановки

lipschitz_second_derivative – L через вторую производную

Реализация принципа из раздела 2: символьные W′ и W″, корни W″ на отрезке, максимум |W′| среди корней и концов отрезка. Если вторая производная не определена или значения не конечны, функция выбрасывает исключение. Для постоянной функции (W″ тождественно нулевая) кандидатами остаются только концы отрезка.

```
def lipschitz_second_derivative(expr, a, b, n_scan=4000):
    """
    L = max |W'(x)| на [a, b].

    Принцип: на отрезке максимум |W'| достигается либо на концах, либо в точках,
    где W'' = 0 (там W' имеет экстремум). Поэтому
      1) берём W'(x) и W''(x) символьно (sympy);
      2) находим корни W''(x) = 0 на [a, b];
      3) считаем |W'| в этих корнях и на концах и берём наибольшее.

    Сетка n_scan нужна только чтобы найти промежутки, где W'' меняет знак;
    сами корни уточняются методом Брента, а L считается по точным значениям W'.
    Если вторая производная не определена (например, |x - 1|) или вычисляется
    нечисловым значением, бросаем исключение - вызывающий код перейдёт на сетку.
    """
    x = sp.Symbol("x")
    f = parse_expr(expr, local_dict={"x": x, "pi": sp.pi, "e": sp.E})
    if f.free_symbols - {x}:
        raise ValueError(f"в выражении есть лишние переменные: {f.free_symbols - {x}}")

    d1 = sp.diff(f, x)
    d2 = sp.diff(d1, x)
    f1 = sp.lambdify(x, d1, "numpy")
    f2 = sp.lambdify(x, d2, "numpy")

    xs = np.linspace(a, b, n_scan)
    v2 = np.broadcast_to(np.asarray(f2(xs), dtype=float), xs.shape)
    if not np.all(np.isfinite(v2)):
        raise ValueError("вторая производная не определена или не конечна на отрезке")

    critical = [a, b]
    if not np.all(v2 == 0.0):                # если W'' тождественно 0, W' постоянна
        for i in range(len(xs) - 1):
            if v2[i] == 0.0:
                critical.append(xs[i])
            elif v2[i] * v2[i + 1] < 0:
                critical.append(brentq(lambda t: float(f2(t)), xs[i], xs[i + 1], xtol=1e-14))

    values = [abs(float(f1(c))) for c in critical]
    if not all(math.isfinite(v) for v in values):
        raise ValueError("первая производная не определена или не конечна в критических точках")
    k = int(np.argmax(values))

    info = {
        "W'": str(d1),
        "W''": str(d2),
        "critical": sorted(set(critical)),
        "argmax": critical[k],
    }
    return values[k], info
```

Рисунок 6 – Оценка константы Липшица через вторую производную

lipschitz_grid и get_L – запасной способ

lipschitz_grid оценивает L по сетке с запасом 1,2. get_L выбирает способ и возвращает вместе с L признак, найдена ли она точно. При ошибке разбора выражения get_L печатает заметное предупреждение и переходит на сетку, а в итоге выводится, что гарантии точности нет.

```python
def lipschitz_grid(W, a, b, n=5000, k=1.2):
    """ запасной вариант: max |ΔW/Δx| по сетке, умноженный на запас k """
    xs = np.linspace(a, b, n)
    ys = np.array([W(t) for t in xs], dtype=float)
    if not np.all(np.isfinite(ys)):
        raise ValueError("на отрезке есть точки, где функция не конечна")
    return float(np.max(np.abs(np.diff(ys) / np.diff(xs)))) * k

def get_L(expr, W, a, b, method="d2"):
    """
    Возвращает (L, описание, exact).
    exact = True только если L найдена через вторую производную и гарантия
    точности eps сохраняется; для сетки и ручного ввода гарантии нет.
    """
    if method == "d2":
        try:
            L, info = lipschitz_second_derivative(expr, a, b)
            return L, f"через 2-ю производную (max|W'| в x = {info['argmax']:.5f})", True
        except Exception as err:
            print("\n" + "!" * 64)
            print("ВНИМАНИЕ: через вторую производную L найти не удалось.")
            print("  Вторая производная не определена или не вычисляется (так бывает,")
            print("  например, у функций с модулем |x - c|, где у W' есть излом).")
            print(f"  Техническая причина: {str(err).splitlines()[0][:70]}")
            print("Перехожу на оценку по сетке с запасом 1.2. Эта оценка приближённая,")
            print("гарантии точности eps в этом режиме нет.")
            print("!" * 64)
    return lipschitz_grid(W, a, b), "по сетке с запасом 1.2 (приближённо)", False
```

Рисунок 7 – Оценка L по сетке и выбор способа

make_function и check_finite – функция из строки и проверка значений

Строка с выражением компилируется и вычисляется в ограниченном окружении, где доступны только математические функции. check_finite до запуска метода проверяет значения на равномерной сетке и в нескольких «неудобных» точках отрезка и не пропускает функции с NaN или бесконечностью.

```python
def make_function(expr):
    """ строка с выражением -> функция W(x) """
    code = compile(expr, "<expr>", "eval")

    def W(x):
        env = dict(SAFE_ENV)
        env["x"] = x
        return eval(code, {"__builtins__": {}}, env)

    return W

def check_finite(W, a, b, n=200):
    """ проверяем, что на отрезке функция вычисляется и значения конечны """
    # равномерная сетка плюс несколько «неудобных» долей отрезка (середина и т.п.)
    extra = [a + (b - a) * q for q in (0.5, 0.1234567, 0.3141592, 0.7182818, 0.8660254)]
    for x in list(np.linspace(a, b, n)) + extra:
        try:
            with np.errstate(all="ignore"):
                v = float(W(x))
        except Exception as err:
            return False, f"не удалось вычислить W({x:.6g}): {err}"
        if not math.isfinite(v):
            return False, f"W({x:.6g}) = {v}: функция не конечна на отрезке"
    return True, ""
```

Рисунок 8 – Преобразование строки в функцию и проверка значений

ask_custom, choose_function, ask_eps, ask_method, ask_L, main – ввод данных

choose_function показывает меню тестовых функций или принимает свою (ask_custom), ask_eps, ask_method и ask_L спрашивают точность и способ получения L (вторая производная, сетка или ручной ввод), main собирает всё вместе.

```
def ask_custom():
    """ своя функция: выражение, отрезок, проверка. Возвращает кортеж или None """
    expr = input("f(x) = ").strip()
    try:
        W = make_function(expr)
    except SyntaxError as err:
        print(f"ошибка в выражении: {err}")
        return None
    try:
        a = float(input("a = "))
        b = float(input("b = "))
    except ValueError:
        print("a и b должны быть числами")
        return None
    if a >= b:
        print("должно быть a < b")
        return None
    ok, msg = check_finite(W, a, b)
    if not ok:
        print(f"функция не подходит: {msg}")
        return None
    return expr, W, a, b, "своя функция"

def choose_function():
    """ меню тестовых функций; возвращает (expr, W, a, b, name) """
    while True:
        print("\nвыберите тестовую функцию:")
        print("-" * 70)
        print(f"{'№':>2}  {'имя':<12} {'W(x)':<35} [a, b]")
        print("-" * 70)
        for i, t in enumerate(TESTS, 1):
            print(f"{i:>2}  {t['name']:<12} {t['expr']:<35} [{t['a']}, {t['b']}]")
        print(f"{len(TESTS) + 1:>2}  своя функция")
        print("-" * 70)

        choice = input("номер: ").strip()
        if not choice.isdigit():
            print("нужно ввести номер")
            continue

        n = int(choice)
        if 1 <= n <= len(TESTS):
            t = TESTS[n - 1]
            return t["expr"], make_function(t["expr"]), t["a"], t["b"], t["name"]

        if n == len(TESTS) + 1:
            custom = ask_custom()
            if custom:
                return custom
            continue

        print("такого номера нет")
```

Рисунок 9 – Меню выбора функции и ввод своей функции

```
def ask_eps():
    while True:
        try:
            eps = float(input("eps = "))
            if eps > 0:
                return eps
            print("eps должно быть положительным")
        except ValueError:
            print("нужно ввести число")

def ask_method():
    print("\nкак получить константу Липшица?")
    print("  1 - через вторую производную (по умолчанию)")
    print("  2 - по сетке с запасом 1.2")
    print("  3 - ввести L вручную")
    return {"2": "grid", "3": "manual"}.get(input("номер: ").strip(), "d2")

def ask_L():
    while True:
        try:
            L = float(input("L = "))
            if L >= 0 and math.isfinite(L):
                return L
            print("L должна быть неотрицательным числом")
        except ValueError:
            print("нужно ввести число")

def main():
    expr, W, a, b, name = choose_function()
    eps = ask_eps()
    method = ask_method()

    if method == "manual":
        L, how, exact = ask_L(), "введена вручную", False
    else:
        try:
            L, how, exact = get_L(expr, W, a, b, method)
        except ValueError as err:
            print(f"не удалось оценить L: {err}")
            return

    print(f"\nфункция: {name}:  {expr}")
    print(f"отрезок: [{a}, {b}],  eps = {eps}")
    print(f"L = {L:.6f}  ({how})")

    try:
        result = pyavsky(W, a, b, eps, L, max_iter=MAX_ITER)
    except ValueError as err:
        print(f"ошибка: {err}")
        return

    print_result(name, result, L, exact)

    title = f"{name}, eps = {eps}"
    if not result["success"]:
        title += "  (ТОЧНОСТЬ НЕ ДОСТИГНУТА)"
    plot_result(W, a, b, result, L, title=title)
```

Рисунок 10 – Ввод точности, выбор способа получения L и точка входа

print_result и plot_result – вывод

print_result печатает статус, L, u*, W(u*), нижнюю оценку, число итераций и время. Если точность не достигнута, выводятся причина и текущая погрешность, а если L приближённая, отдельной строкой сообщается об отсутствии гарантии. plot_result строит график: функцию, ломаную, галочки, верхние вершины и найденный минимум, справа дополнительно увеличенная окрестность минимума.

```
def print_result(name, result, L, exact=True):
    width = 64
    print("\n" + "=" * width)
    print(f"результаты ({name}):")
    print(f"  статус           = {STATUS_TEXT[result['status']]}")
    print(f"  L                = {L:.6f}")
    print(f"  u*               = {result['u_min']:.10f}")
    print(f"  W(u*)            = {result['W_min']:.10f}")
    if math.isfinite(result["lower_bound"]):
        print(f"  нижняя оценка    = {result['lower_bound']:.10f}")
    print(f"  итераций         = {result['iterations']}")
    print(f"  время            = {result['time']:.4f} сек")
    print("-" * width)
    if result["success"]:
        if exact:
            print("  гарантия: найденный минимум отличается от истинного меньше чем на eps")
        else:
            print("  гарантии точности нет: L приближённая, она может быть меньше настоящей")
    else:
        print("!!! РЕЗУЛЬТАТ ПРИБЛИЖЁННЫЙ, ТОЧНОСТЬ eps НЕ ДОСТИГНУТА")
        print(textwrap.fill(f"причина: {result['message']}", width=62,
                            initial_indent="!!! ", subsequent_indent="!!!   "))
        if result["status"] == "max_iter":
            print(f"!!! на данный момент погрешность не больше {result['W_min'] - result['lower_bound']:.3e}"
                  " (если L верная)")
        if result["status"] == "L_too_small":
            print("!!! выход: увеличьте L (или выберите оценку через 2-ю производную)")
    print("=" * width)
```

Рисунок 11 – Функция print_result

4. Результаты работы

Расчёты выполнены с константой Липшица, найденной через вторую производную. На графиках слева весь отрезок: синяя линия — W(x), красный пунктир — итоговая ломаная, тонкие зелёные линии — вспомогательные функции g_i, зелёные точки — верхние вершины, красная точка — найденный минимум. Справа — увеличенная окрестность найденного минимума.

Таблица 2 – Итоги расчётов

Функция	[a, b]	ε	L	u*	W(u*)	Нижняя оценка	Число итераций	Время, с
Растригин	[−2; 3]	0,001	67,3327	1,000007	$8{,}6\cdot10^{-9}$	$-9{,}9\cdot10^{-4}$	357	0,0159
Из задания	[−2; 2]	0,01	4,1416	−2,000000	−1,999995	−2,000000	11	0,0001
Гриванк	[−20; 20]	0,001	1,0071	0,000000	0	$-8{,}3\cdot10^{-4}$	208	0,0057

4.1. Функция Растригина

$W(x) = 10 + (x - 1)^2 - 10 \cdot \cos(2\pi x)$, отрезок $[-2; 3]$, $\varepsilon = 0{,}001$. На отрезке несколько локальных минимумов, глобальный — в точке $x = 1$, где $W = 0$. Найдено $u^* = 1{,}000007$, $W(u^*) = 8{,}6 \cdot 10^{-9}$. Видно, что точки сгущаются в окрестности глобального минимума, а около остальных локальных минимумов вычислений заметно меньше: ломаная там быстро поднимается выше текущего лучшего значения.

```
выберите тестовую функцию:
------------------------------------------------------------------------
 №  имя          W(x)                                  [a, b]
------------------------------------------------------------------------
 1  Растригин    10 + (x - 1)**2 - 10*cos(2*pi*x)      [-2.0, 3.0]
 2  Из задания   x + sin(3.14159*x)                    [-2.0, 2.0]
 3  Гриванк      1 + x**2/4000 - cos(x)                [-20.0, 20.0]
 4  своя функция
------------------------------------------------------------------------
номер: 1
eps = 0.001

как получить константу Липшица?
  1 - через вторую производную (по умолчанию)
  2 - по сетке с запасом 1.2
  3 - ввести L вручную
номер: 1

функция: Растригин:  10 + (x - 1)**2 - 10*cos(2*pi*x)
отрезок: [-2.0, 3.0],  eps = 0.001
L = 67.332659  (через 2-ю производную (max|W'| в x = -1.25081))

============================================================
результаты (Растригин):
  статус           = точность достигнута
  L                = 67.332659
  u*               = 1.0000065907
  W(u*)            = 0.0000000086
  нижняя оценка    = -0.0009929417
  итераций         = 357
  время            = 0.0159 сек
------------------------------------------------------------
  гарантия: найденный минимум отличается от истинного меньше чем на eps
============================================================
```

Рисунок 12 – Запуск программы, функция Растригина

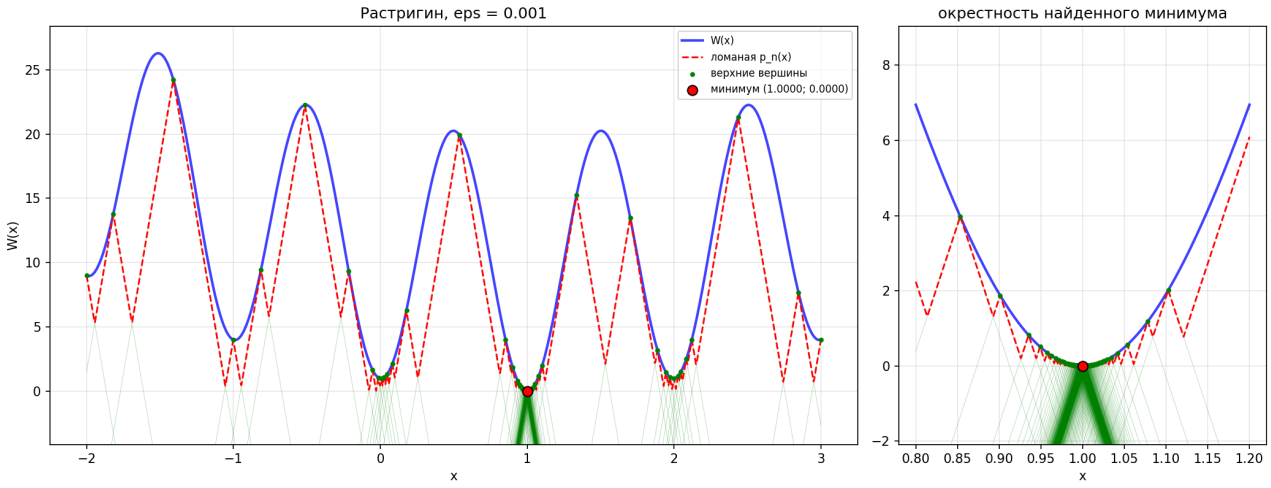


Рисунок 13 – График, функция Растригина, ε = 0,001

4.2. Функция из задания

W(x) = x + sin(3,14159·x), отрезок [−2; 2], ε = 0,01. Здесь два внутренних локальных минимума (x ≈ −0,60 и x ≈ 1,40), но глобальный минимум лежит на левом конце: u* = −2, W(u*) ≈ −2. Метод справился за 11 итераций.

```
выберите тестовую функцию:
------------------------------------------------------------------------
 №  имя          W(x)                                    [a, b]
------------------------------------------------------------------------
 1  Растригин    10 + (x - 1)**2 - 10*cos(2*pi*x)    [-2.0, 3.0]
 2  Из задания   x + sin(3.14159*x)                  [-2.0, 2.0]
 3  Гриванк      1 + x**2/4000 - cos(x)              [-20.0, 20.0]
 4  своя функция
------------------------------------------------------------------------
номер: 2
eps = 0.01

как получить константу Липшица?
  1 - через вторую производную (по умолчанию)
  2 - по сетке с запасом 1.2
  3 - ввести L вручную
номер: 1

функция: Из задания:  x + sin(3.14159*x)
отрезок: [-2.0, 2.0],  eps = 0.01
L = 4.141590  (через 2-ю производную (max|W'| в x = 0.00000))

============================================================
результаты (Из задания):
  статус          = точность достигнута
  L               = 4.141590
  u*              = -2.0000000000
  W(u*)           = -1.9999946928
  нижняя оценка   = -1.9999947487
  итераций        = 11
  время           = 0.0001 сек
------------------------------------------------------------
  гарантия: найденный минимум отличается от истинного меньше чем на eps
============================================================
```

Рисунок 14 – Запуск программы, функция из задания

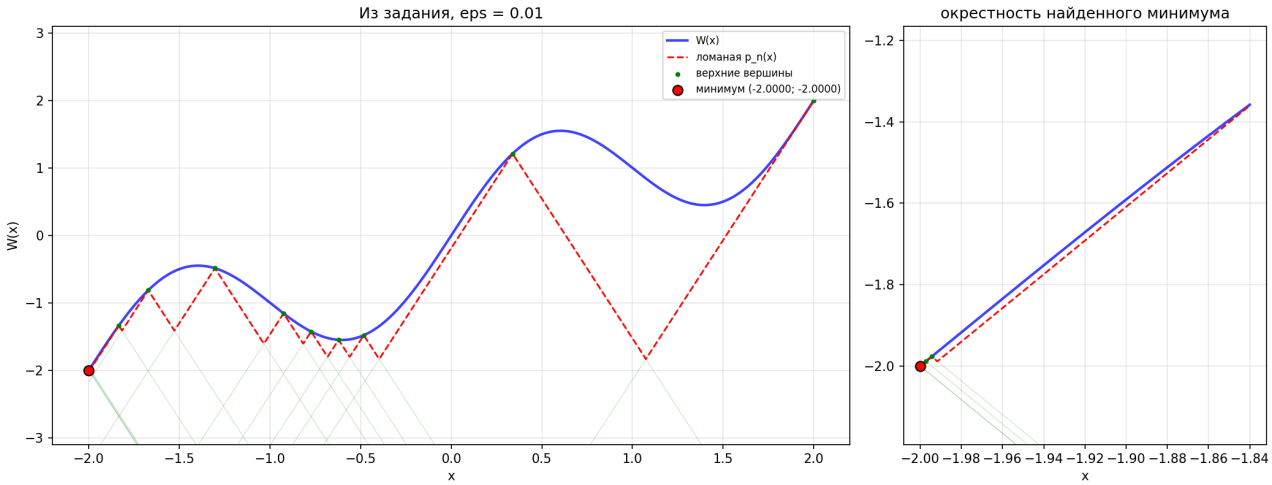


Рисунок 15 – График, функция из задания, ε = 0,01

4.3. Функция Гриванка (одномерная)

W(x) = 1 + x²/4000 − cos(x), отрезок [−20; 20], ε = 0,001. На отрезке около шести локальных минимумов с почти одинаковой глубиной, глобальный — в нуле. Найдено u* = 0, W(u*) = 0.

```
выберите тестовую функцию:
------------------------------------------------------------------------
 №  имя          W(x)                                    [a, b]
------------------------------------------------------------------------
 1  Растригин    10 + (x - 1)**2 - 10*cos(2*pi*x)    [-2.0, 3.0]
 2  Из задания   x + sin(3.14159*x)                  [-2.0, 2.0]
 3  Гриванк      1 + x**2/4000 - cos(x)              [-20.0, 20.0]
 4  своя функция
------------------------------------------------------------------------
номер: 3
eps = 0.001

как получить константу Липшица?
  1 - через вторую производную (по умолчанию)
  2 - по сетке с запасом 1.2
  3 - ввести L вручную
номер: 1

функция: Гриванк:  1 + x**2/4000 - cos(x)
отрезок: [-20.0, 20.0],  eps = 0.001
L = 1.007069  (через 2-ю производную (max|W'| в x = -14.13767))

============================================================
результаты (Гриванк):
  статус          = точность достигнута
  L               = 1.007069
  u*              = 0.0000000000
  W(u*)           = 0.0000000000
  нижняя оценка   = -0.0008325653
  итераций        = 208
  время           = 0.0057 сек
------------------------------------------------------------
  гарантия: найденный минимум отличается от истинного меньше чем на eps
============================================================
```

Рисунок 16 – Запуск программы, функция Гриванка

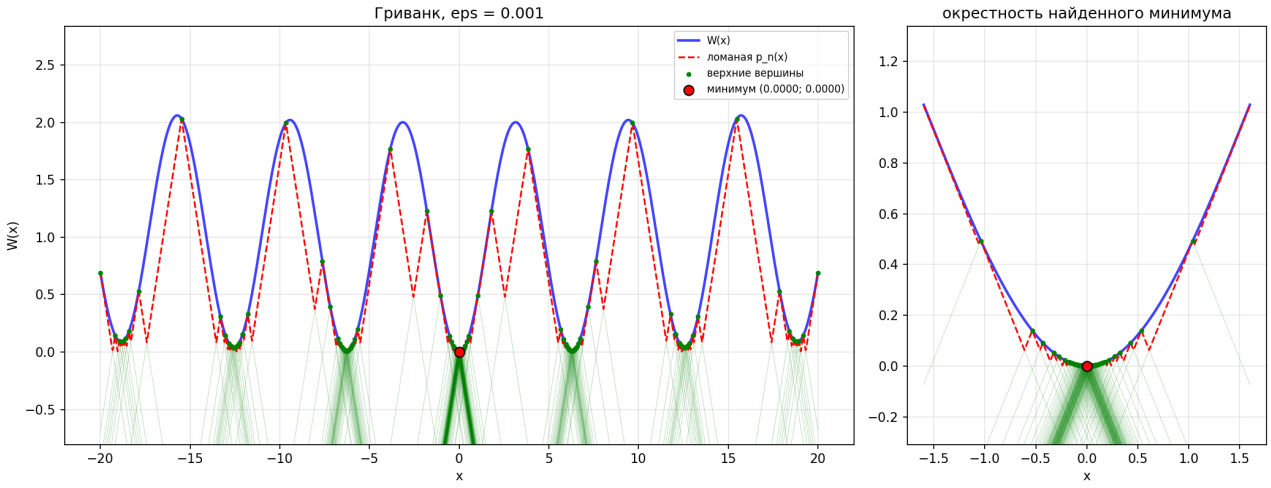


Рисунок 17 – График, функция Гриванка, ε = 0,001

5. Особые случаи и проверки

Ниже показано, как программа ведёт себя в ситуациях, когда метод нельзя довести до результата с гарантией. Во всех этих случаях программа не выдаёт результат как успешный, а явно сообщает причину.

5.1. Константа L занижена

Для функции Растригина настоящая константа Липшица равна 67,33, а вручную введено L = 10. Метод видит, что между двумя соседними точками функция растёт круче, чем допускает L (наклон 11,77 > 10), останавливается на пятой итерации и пишет, что точность не достигнута. Найденный наклон одновременно является оценкой снизу для настоящей L, поэтому программа сообщает и его.

```
(меню выбора функции)
номер: 1
eps = 0.001

как получить константу Липшица?
  1 - через вторую производную (по умолчанию)
  2 - по сетке с запасом 1.2
  3 - ввести L вручную
номер: 3
L = 10

функция: Растригин:  10 + (x - 1)**2 - 10*cos(2*pi*x)
отрезок: [-2.0, 3.0],  eps = 0.001
L = 10.000000  (введена вручную)

================================================================
результаты (Растригин):
  статус           = ТОЧНОСТЬ НЕ ДОСТИГНУТА (L занижена)
  L                = 10.000000
  u*               = 3.0000000000
  W(u*)            = 4.0000000000
  нижняя оценка    = 1.4025680544
  итераций         = 5
  время            = 0.0001 сек
----------------------------------------------------------------
!!! РЕЗУЛЬТАТ ПРИБЛИЖЁННЫЙ, ТОЧНОСТЬ eps НЕ ДОСТИГНУТА
!!! причина: L = 10 занижена: на отрезке [2.17813; 2.74026]
!!!   наклон функции 11.7702 больше L. Настоящая константа
!!!   Липшица не меньше 11.7702
!!! выход: увеличьте L (или выберите оценку через 2-ю производную)
================================================================
```

Рисунок 18 – Запуск с заниженной L

5.2. Исчерпан лимит итераций

При лимите 50 итераций точность 0,001 не достигается. Программа показывает статус «лимит итераций», причину и текущую гарантированную погрешность. Результат при этом выводится, но не выдаётся за итог с заданной точностью, а на графике заголовок помечен предупреждением.

```
(меню выбора функции)
номер: 1
eps = 0.001

как получить константу Липшица?
  1 - через вторую производную (по умолчанию)
  2 - по сетке с запасом 1.2
  3 - ввести L вручную
номер: 1

функция: Растригин:  10 + (x - 1)**2 - 10*cos(2*pi*x)
отрезок: [-2.0, 3.0],  eps = 0.001
L = 67.332659  (через 2-ю производную (max|W'| в x = -1.25081))

================================================================
результаты (Растригин):
  статус           = ТОЧНОСТЬ НЕ ДОСТИГНУТА (лимит итераций)
  L                = 67.332659
  u*               = 0.9993650080
  W(u*)            = 0.0000799945
  нижняя оценка    = -0.1284688939
  итераций         = 50
  время            = 0.0004 сек
----------------------------------------------------------------
!!! РЕЗУЛЬТАТ ПРИБЛИЖЁННЫЙ, ТОЧНОСТЬ eps НЕ ДОСТИГНУТА
!!! причина: исчерпан лимит 50 итераций, достигнута только
!!!   W(u_n) - p = 1.285e-01 (нужно < 0.001)
!!! на данный момент погрешность не больше 1.285e-01 (если L верная)
================================================================
```

Рисунок 19 – Запуск с лимитом 50 итераций

5.3. Постоянная функция

Для W(x) = 5 вторая производная даёт L = 0, а формула нижней вершины содержит деление на 2L. Это отдельная ветка: если значения на концах и в середине отрезка совпадают, функция постоянна и минимум достигается в любой точке, поэтому возвращается левый конец за 0 итераций. Если же L = 0, а функция не постоянна, программа сообщает, что L занижена.

```
(меню выбора функции)
номер: 4
f(x) = 5
a = -1
b = 3
eps = 0.01

как получить константу Липшица?
  1 - через вторую производную (по умолчанию)
  2 - по сетке с запасом 1.2
  3 - ввести L вручную
номер: 1

функция: своя функция:  5
отрезок: [-1.0, 3.0],  eps = 0.01
L = 0.000000  (через 2-ю производную (max|W'| в x = -1.00000))

================================================================
результаты (своя функция):
  статус           = функция постоянна
  L                = 0.000000
  u*               = -1.0000000000
  W(u*)            = 5.0000000000
  нижняя оценка    = 5.0000000000
  итераций         = 0
  время            = 0.0001 сек
----------------------------------------------------------------
  гарантия: найденный минимум отличается от истинного меньше чем на eps
================================================================
```

Рисунок 20 – Запуск для постоянной функции

5.4. Функция без второй производной

У функции |x − 1| вторая производная содержит разрыв, поэтому L через W″ найти нельзя. Программа печатает выделенное предупреждение, переходит на оценку по сетке (получается 1,2 при настоящем значении 1) и в итоге отдельной строкой пишет, что гарантии точности нет.

```
(меню выбора функции)
номер: 4
f(x) = abs(x - 1)
a = -2
b = 3
eps = 0.01

как получить константу Липшица?
  1 - через вторую производную (по умолчанию)
  2 - по сетке с запасом 1.2
  3 - ввести L вручную
номер: 1

!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
ВНИМАНИЕ: через вторую производную L найти не удалось.
  Вторая производная не определена или не вычисляется (так бывает,
  например, у функций с модулем |x - c|, где у W' есть излом).
  Техническая причина: Unsupported by <class 'sympy.printing.numpy.NumPyPrinter'>: <class 'sy
Перехожу на оценку по сетке с запасом 1.2. Эта оценка приближённая,
гарантии точности eps в этом режиме нет.
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!

функция: своя функция:  abs(x - 1)
отрезок: [-2.0, 3.0],  eps = 0.01
L = 1.200000  (по сетке с запасом 1.2 (приближённо))

============================================================
результаты (своя функция):
  статус          = точность достигнута
  L               = 1.200000
  u*              = 0.9999959812
  W(u*)           = 0.0000040188
  нижняя оценка   = -0.0012779707
  итераций        = 9
  время           = 0.0001 сек
------------------------------------------------------------
  гарантии точности нет: L приближённая, она может быть меньше настоящей
============================================================
```

Рисунок 21 – Запуск для функции |x − 1|

5.5. Значения, которые не являются конечными числами

Перед запуском значения функции проверяются на всём отрезке. Для ln(x) на [−1; 2] получается NaN, и программа не принимает такую функцию и просит ввести другую. Если NaN или бесконечность появится уже в ходе итераций, метод остановится со статусом «функция не конечна».

```
(меню выбора функции)
номер: 4
f(x) = log(x)
a = -1
b = 2
функция не подходит: W(-1) = nan: функция не конечна на отрезке

(меню выбора функции)
```

Рисунок 22 – Отказ принять функцию с NaN

Выводы

В работе реализован метод ломаных для поиска глобального минимума липшицевой функции и проверен на трёх функциях с несколькими локальными минимумами. Во всех случаях найден глобальный минимум с заданной точностью, а нижняя оценка $p_{n-1}(u_n)$ подтверждает, что отклонение от истинного значения не больше ε.

Сначала я не сразу понял, откуда берётся следующая точка: казалось, что нужно искать минимум ломаной по всему отрезку. После того как стало ясно, что он всегда в пересечении галочек соседних вершин, алгоритм сильно упростился, и значение ломаной в этой точке тоже считается по формуле, без перебора. Критерий остановки $W(u_n) - p_{n-1}(u_n) < \varepsilon$ оказался удобнее, чем просто сравнение соседних значений, потому что он сразу даёт гарантию точности.

Самым важным оказался выбор L. Оценка по сетке с запасом работает, но завышает константу и тратит лишние итерации (на тестах на 25–35% больше). Вычисление через вторую производную даёт точное значение: максимум $|W'|$ достигается на концах или в нулях W'', значит перебирать нужно всего несколько точек. Ограничение способа в том, что он требует дифференцируемой функции и символьного разбора выражения; для негладких функций приходится возвращаться к оценке по сетке, поэтому оба варианта оставлены в программе.

Дополнительно в программе обработаны ситуации, когда метод нельзя довести до результата с гарантией: заниженная L, исчерпанный лимит итераций, постоянная функция, отсутствие второй производной и нечисловые значения (раздел 5). Главное правило, которое я для себя вывел: об успехе можно сообщать только тогда, когда выполнено условие точности и нет признаков, что L занижена.

Код программы: https://github.com/AbdelazizBenChamekh/Optimization_Methods/tree/main/Lab2